



CSc 453

COMPILERS AND
SYSTEMS
SOFTWARE

Version: 2026-02-28

MIDTERM REVIEW SPRING 2026

Problem 1

Points: 10/100

- Use Thompson's Construction to build an NFA from this regular expression

$((ab)^* | c^+)$

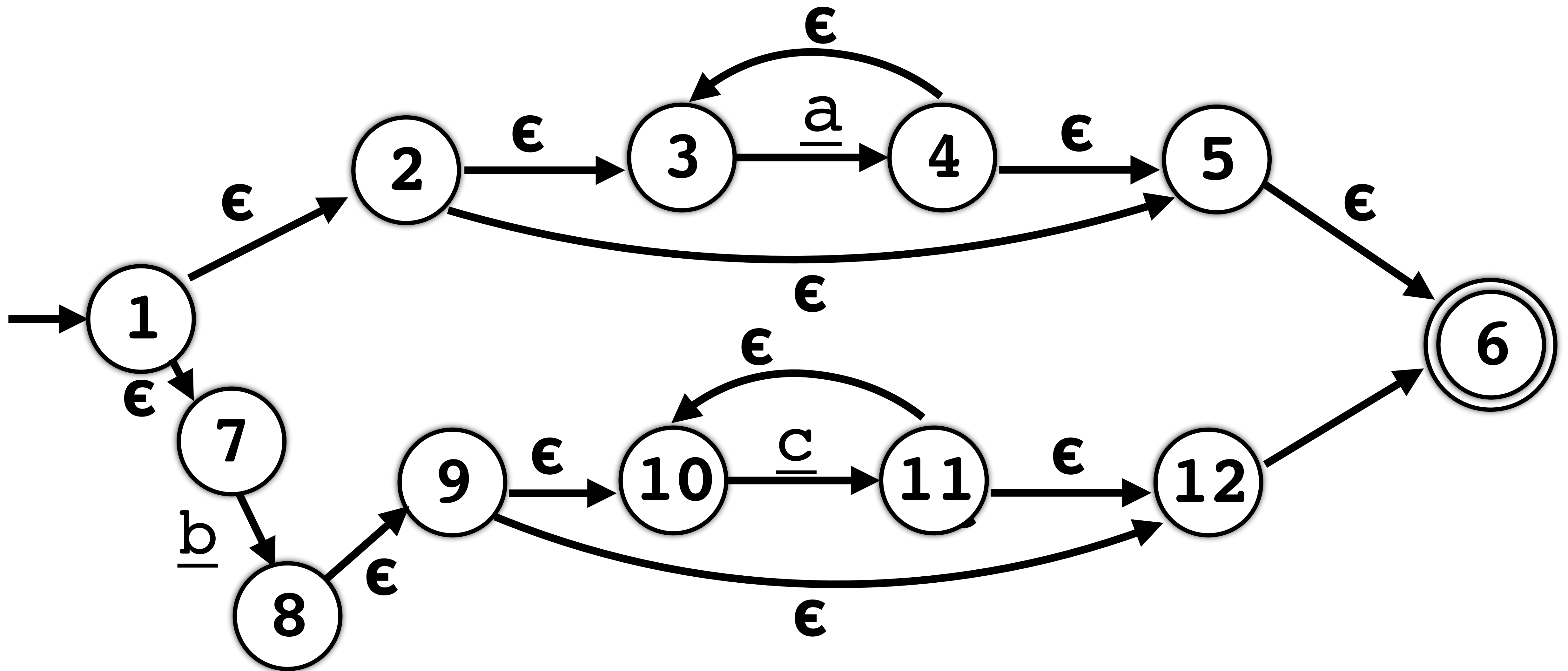
- I have fully parenthesized the expression so that you don't have to worry about the precedence of regular expression operators.

Problem 2 (a)

Points: 20/100

- Consider the NFA on the next page.
 1. Compute $\epsilon\text{-closure}(\{s\})$ for all states s .
 2. Compute $\text{move}(\{8\}, \underline{b})$.
 3. Use the subset construction algorithm to convert this NFA into a DFA.
 - Make sure that you label the states in the DFA A, B, C , etc.
 - For every DFA state list the corresponding set of states in the NFA. This is so that I know that you actually used the subset construction algorithm.

Task 2 (b)



Problem 3

Points: 20/100

•Consider this grammar:

1. Pick a sentence generated by this grammar and show a leftmost derivation
2. Pick a sentence generated by this grammar and show the parse tree for a leftmost derivation
3. Show the FIRST set for all non-terminal symbols
4. Show the FOLLOW set for all non-terminal symbols

S	→	<u>u</u>	B	D	<u>z</u>
B	→	<u>B</u>	<u>v</u>		<u>w</u>
D	→	E	<u>F</u>		
E	→	<u>y</u>		ε	
F	→	<u>x</u>		ε	

Problem 4 (a)

Points: 30/100

- Consider this grammar:

1. Show the predictive parse table for this grammar.

$S \rightarrow A B C$

$A \rightarrow \underline{x}$

$A \rightarrow \underline{y} A$

$B \rightarrow \epsilon$

$B \rightarrow \underline{z} B$

$C \rightarrow \underline{w}$

$\text{FIRST}(S) = \{\underline{x}, \underline{y}\}$

$\text{FIRST}(A) = \{\underline{x}, \underline{y}\}$

$\text{FIRST}(B) = \{\underline{z}, \epsilon\}$

$\text{FIRST}(C) = \{\underline{w}\}$

$\text{FOLLOW}(S) = \{\$ \}$

$\text{FOLLOW}(A) = \{\underline{z}, \underline{w}\}$

$\text{FOLLOW}(B) = \{\underline{w}\}$

$\text{FOLLOW}(C) = \{\$ \}$

Problem 4 (b)

2. Show a pseudocode recursive descent parser for this grammar.

- You can assume the existence of functions `match(kind)` and `lookahead(kind)`:

```
void match(TokenKind t) {  
    if (curr_tok==t)  
        curr_tok=next_token();  
    else  
        syntax error  
}
```

```
void lookahead(TokenKind t){  
    return (curr_tok==t);  
}
```

Problem 5

Points: 10/100

- Left factor this grammar:

$$\begin{array}{l} S \rightarrow \underline{b} S S \underline{a} \underline{a} S \\ S \rightarrow \underline{b} S S \underline{a} S \underline{b} \\ S \rightarrow \underline{b} S \underline{b} \\ S \rightarrow \underline{a} \end{array}$$

Problem 6

Points: 10/100

- Remove left recursion from this grammar:

$$S \rightarrow (L)$$
$$S \rightarrow \underline{a}$$
$$L \rightarrow L \underline{\wedge} S$$
$$L \rightarrow S$$

Problem 7

Points: 10/100

- Rewrite this grammar so that it encodes the correct precedence and associativity for addition and multiplication:

$$\begin{array}{lcl} E & ::= & E \ + \ E \\ & & E \ _ \ E \\ & & \underline{id} \\ & & \underline{num} \end{array} \quad \begin{array}{l} | \\ | \\ | \\ \end{array}$$

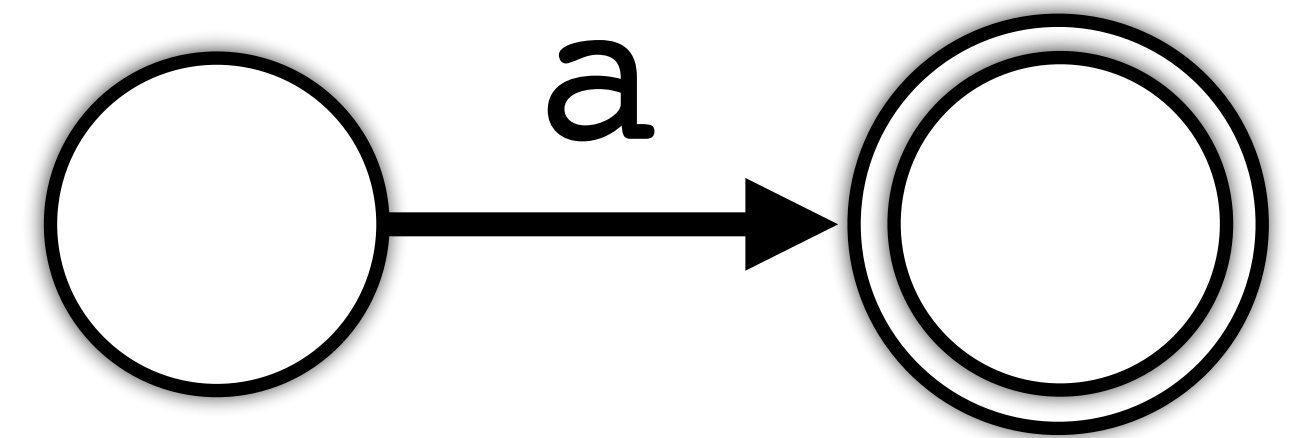
- Do not remove left recursion.



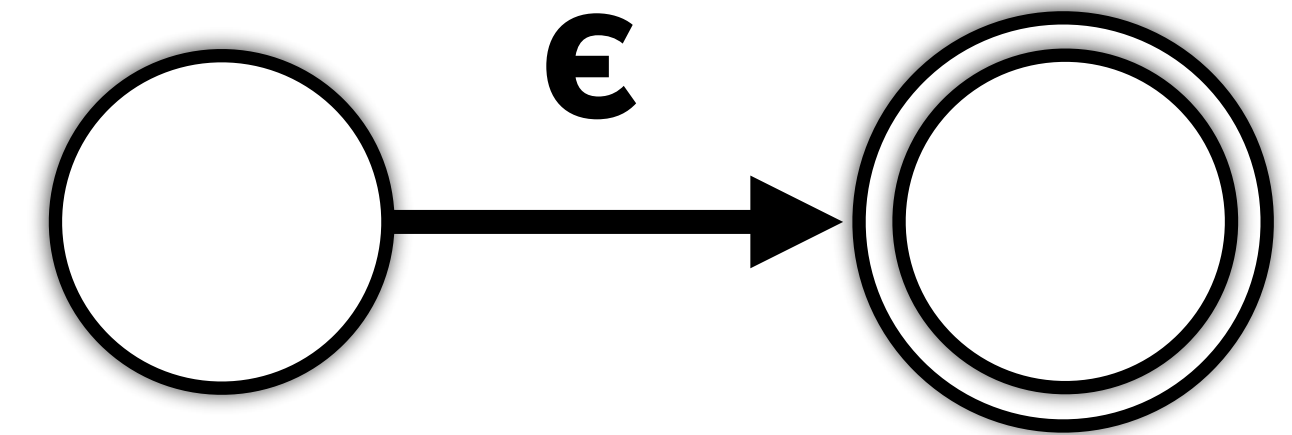
DEFINITIONS AND ALGORITHMS

Thompson's Construction - Characters

The RE matching 'a' →

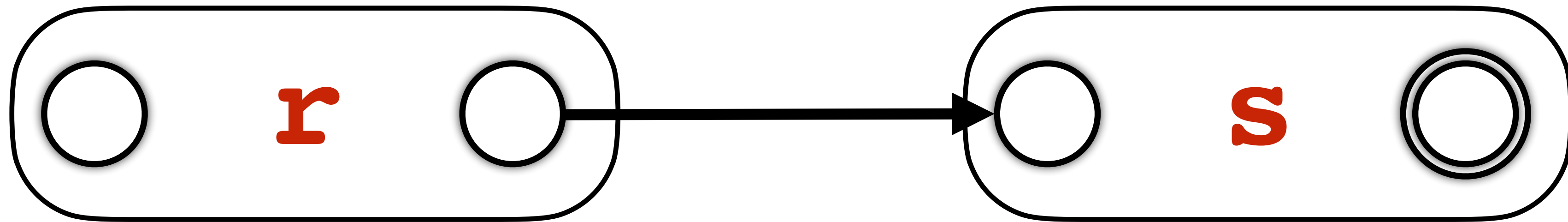


The RE matching ϵ →



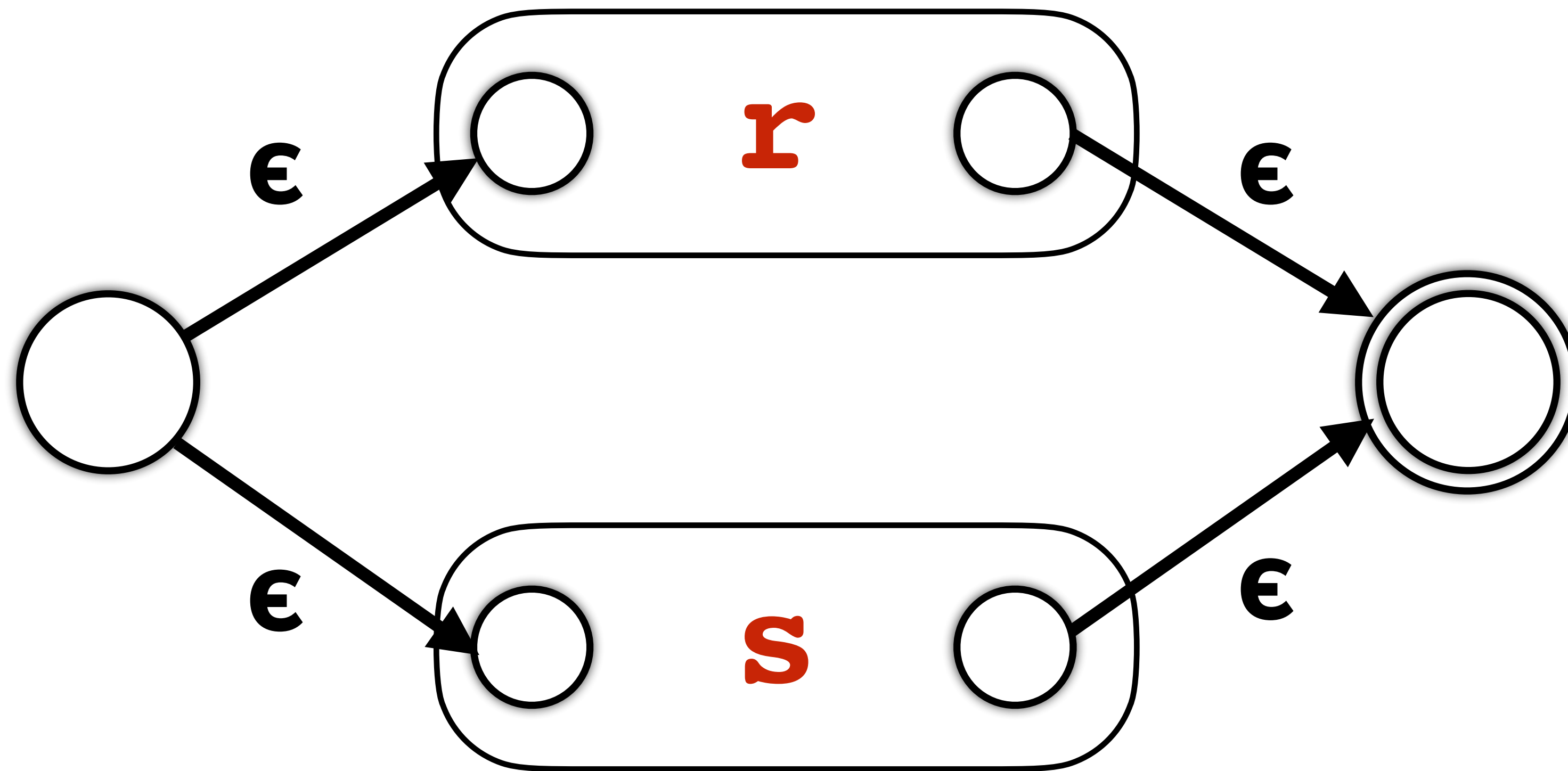
Thompson's Construction - Concatenation

The regular expression **r** followed by **s** translates into



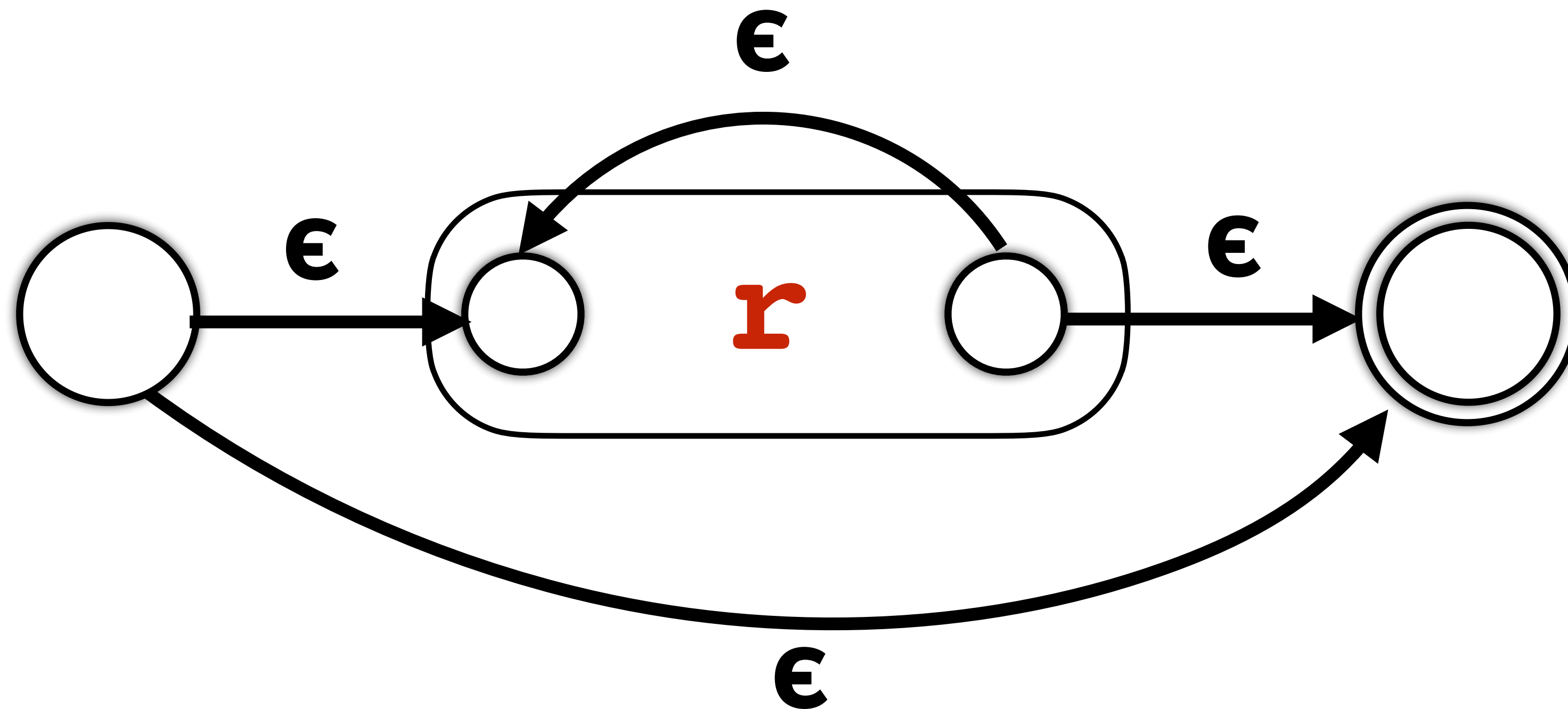
Thompson's Construction - Alternation

The regular expression $r \mid s$ translates into



Thompson's Construction - Repetition

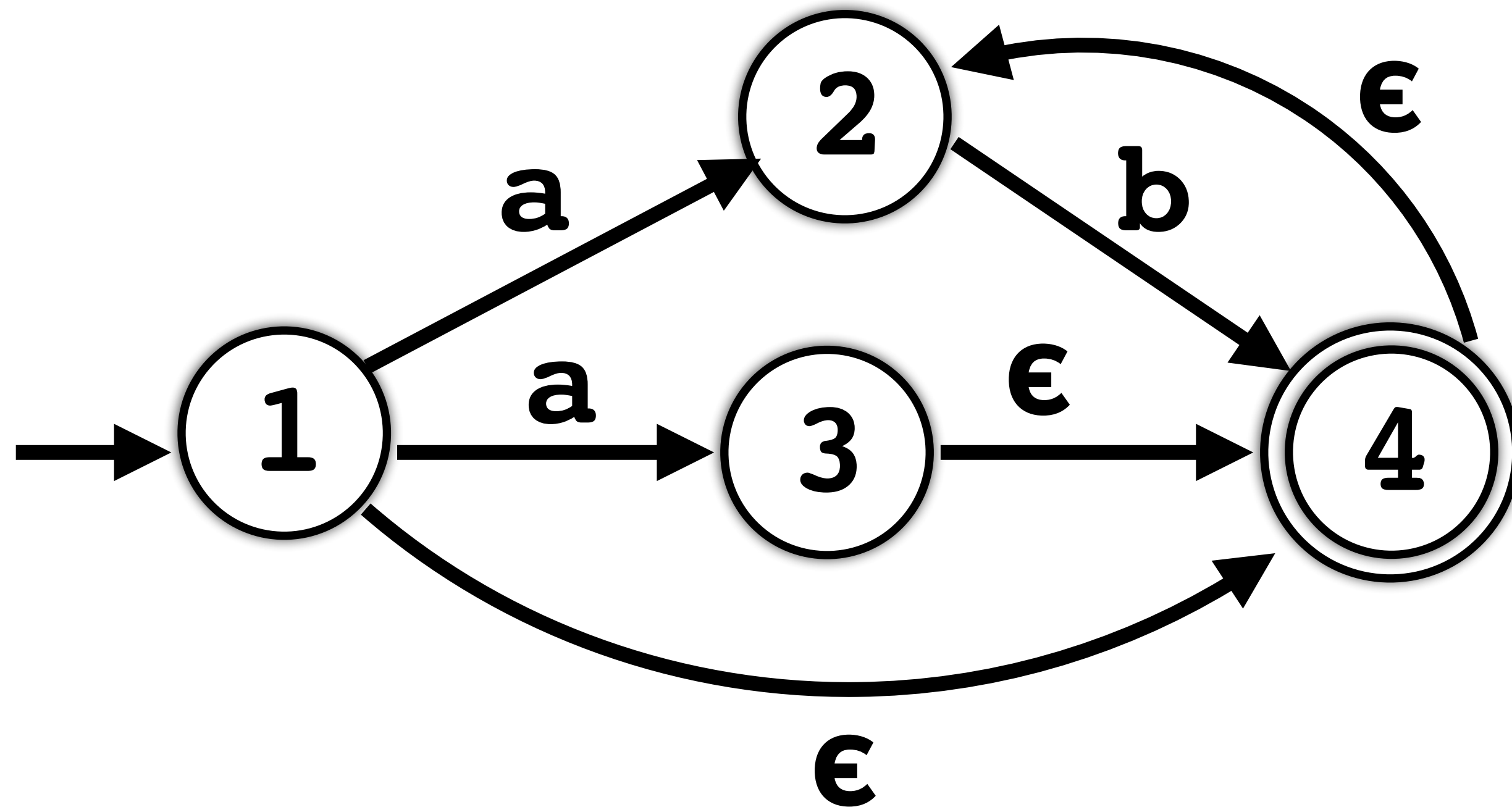
The regular expression r^* translates into




```
procedure  $\epsilon$ -closure(T)
    push all states in T onto stack
    C := T
    while stack is not empty do
        t := pop(stack)
        for each edge  $t \xrightarrow{\epsilon} u$  do
            if u is not in C then
                C := C  $\cup$  u
                push(stack, u)
    return C
```

$\text{move}(T, a)$

- $\text{move}(T, a)$ is the set of NFA states to which there is a transition on input symbol a from some NFA state $s \in T$.



- $\text{move}(\{1\}, a) = \{2, 3\}$
- $\text{move}(\{2, 3\}, b) = \{4\}$

```

procedure SubsetConstruction(NFA N)
    Dstates := { $\epsilon$ -closure( $s_0$ )}
    Dtrans := {}
    repeat
        T := an unexplored state in Dstates
        for each input symbol a do
            U :=  $\epsilon$ -closure(move(T, a))
            if U is not in Dstates then
                Dstates := Dstates  $\cup$  U
                Dtrans := Dtrans  $\cup$  (T  $\xrightarrow{a}$  U)
    until all states have been explored
    return (Dstates, Dtrans)

```


Computing FIRST Sets (with $A \rightarrow \epsilon$ rules)

1. **FOR** *each non-terminal* A **DO** $\text{FIRST}(A) = \{\}$
2. **FOR** *each terminal* t **DO** $\text{FIRST}(t) = \{t\}$
3. **FOR** *each production* $A \rightarrow \epsilon$ **DO** $\text{FIRST}(A) = \{\epsilon\}$
4. **REPEAT** until no more changes:
 - FOR** *each production* $A \rightarrow Y_1 \dots Y_k$ except $A \rightarrow \epsilon$ **DO**
 $\text{FIRST}(A) \cup= \text{FIRST}(Y_1) - \{\epsilon\};$
FOR $i = 1$ to $k-1$ **DO**
 IF ϵ is in $\text{FIRST}(Y_1) \wedge \dots \wedge \epsilon$ is in $\text{FIRST}(Y_i)$ **THEN**
 $\text{FIRST}(A) \cup= \text{FIRST}(Y_{i+1}) - \{\epsilon\};$
 - IF** ϵ is in $\text{FIRST}(Y_1) \wedge \dots \wedge \epsilon$ is in $\text{FIRST}(Y_k)$ **THEN**
 $\text{FIRST}(A) \cup= \{\epsilon\};$

Computing FOLLOW Sets

1. **FOR** each non-terminal A **DO** $\text{FOLLOW}(A) = \{\}$

2. $\text{FOLLOW}(S) = \{\$ \}$

REPEAT until no more changes:

3. **FOR** each production $A \rightarrow \alpha B \beta$ **DO**

$\text{FOLLOW}(B) \cup= (\text{FIRST}(\beta) - \{\epsilon\})$

4. **FOR** each production $A \rightarrow \alpha B$ **DO**

$\text{FOLLOW}(B) \cup= \text{FOLLOW}(A)$

5. **FOR** each production $A \rightarrow \alpha B \beta$ **WHERE** ϵ is in $\text{FIRST}(\beta)$ **DO**

$\text{FOLLOW}(B) \cup= \text{FOLLOW}(A)$

BUILDING THE PARSE TABLE

FOR each production $A \rightarrow \alpha$ **DO**

FOR each terminal x in $\text{FIRST}(\alpha)$ **DO**

$M[A, x] \cup= \{A \rightarrow \alpha\}$

IF ϵ is in $\text{FIRST}(\alpha)$ **THEN**

FOR each terminal x in $\text{FOLLOW}(A)$ **DO**

$M[A, x] \cup= \{A \rightarrow \alpha\}$

IF ϵ is in $\text{FIRST}(\alpha)$ **AND** $\$$ is in $\text{FOLLOW}(A)$ **THEN**

$M[A, \$] \cup= \{A \rightarrow \alpha\}$

FOR all undefined entries $M[A, x]$ **DO**

$M[A, x] := \text{error}$

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA